AI for Lunar Olivine Research

LLM APIs

The Moon is my beautiful impossibility.

Ping Wang

April 2026

Tutorial • Colab Notebooks • Lunar Petrographic Exhibit

This guide is sequenced so that abstract concepts (what an API is) come before concrete tooling (calling OpenAI), and cloud-based tools come before local deployment (running Qwen on local machines), which carries the heaviest setup burden.

Contents

1. What is an API?
2. The request/response cycle
3. HTTP methods
4. Status codes
5. JSON
6. Hugging Face ecosystem
7. Running Qwen locally

1 What is an API?

A simple way to understand an API is to think about a restaurant menu. When you sit down at a restaurant, the menu tells you what you can order. An API works in a similar way. The kitchen is the server-side implementation you might never see. The crucial insight is: **an API is a contract**. The client might not know how the kitchen works, only what the menu promises.

Server-side refers specifically to code that runs on a server rather than in the user's browser or device. Backend refers to the entire part of an application that runs behind the scenes. Backend can include things beyond the server itself, such as databases, authentication systems, etc. The API, server-side code, and database are typically considered the backend. The frontend is what the user sees. The database stores data. The backend, or server-side, is the part of the application that processes requests and provides data to the frontend.

An API (Application Programming Interface) is a way for one software to ask another software for data or services. Using the restaurant analogy:

- The customer is the frontend (website or app).
- The kitchen is the backend (server).
- The menu is the API.

The customer doesn't walk into the kitchen and cook food themselves. Instead, they use the menu to see what they can request and how to request it.

Similarly, a frontend doesn't directly access a database. Instead, it sends requests through an API:

$$\text{Frontend} \rightarrow \text{API} \rightarrow \text{Backend}$$

For example, when you open a weather app:

- The app asks the API: "What's the weather in Boston?"
- The API receives the request.
- The backend gathers the weather data.
- The API returns the result to the app.

- The app displays it to you.

A simple API request might look like:

```
GET /weather?city=Boston
```

And the API might respond with:

```
{
  "temperature": 72,
  "condition": "Sunny"
}
```

So while the backend does the work, the API is the interface that defines how other software can communicate with the backend. An API defines **structured, machine-readable** interactions. A human can interpret a vague menu request; a server cannot.

2 The request/response cycle

`Client` → `Request` → `Server` → `Response` → `Client`

- `Request`: method, URL, headers, and optional body
- `Response`: status code, headers, and body

3 HTTP methods

CRUD mapping:

- GET (read)
- POST (create)
- PUT/PATCH (update)
- DELETE (remove)

4 Status codes

- 2xx: success
- 3xx: redirection
- 4xx: You (the client) made a mistake.
- 5xx: The server made a mistake.

Particularly, you'll see:

- 400: malformed request
- 401: bad/missing key
- 403: key valid but not permitted
- 404/429: rate limited — they **will* see this with AI APIs**.

5 JSON

JSON is text in strict format. Rules of JSON:

- double quotes only
- no comments
- no trailing commas
- `'None'`, `'true'`/`'false'` lowercase

First API call:

```
import requests
response = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params={"latitude": 37.34, "longitude": -121.89,    # San Jose
            "current": "temperature_2m,wind_speed_10m"},
)

print("Status:", response.status_code)
print("Final URL:", response.url)
data = response.json()
print("Temperature:", data["current"]["temperature_2m"], "\circC")
print("Wind:", data["current"]["wind_speed_10m"], "km/h")
```

6 Hugging Face Ecosystem

Hugging Face started as a chatbot company targeting teens, but what it became is far more important: the central **public square** of machine learning. Its core product, the Hugging Face Hub, is like GitHub for models. Just as GitHub hosts code repositories that anyone can browse, clone, and build upon, the Hub hosts over millions of model repositories, each containing trained weights, configuration files, and documentation, alongside hundreds of thousands of datasets and interactive demos called Spaces. Under the hood, every model on the Hub literally is a Git repository, which means the familiar mechanics of version control — commits, branches, diffs, and community pull requests — apply to multi-gigabyte files of learned parameters just as they apply to source code.

Hugging Face's contribution is the infrastructure and, just as importantly, the conventions. Chief among these conventions is the model card: the documentation page attached to every model, which is supposed to state what the model was trained on, what it is intended for, what it should not be used for, what license governs it, and what its known failure modes are. Learning to read a model card critically — including noticing what a card conspicuously fails to say — is one of the most transferable professional skills in modern machine learning.

Hugging Face maintains the open-source software that animates the ecosystem, most famously the `transformers` library, which provides a uniform Python interface for downloading and running nearly any model on the Hub, so that the same few lines of code can drive a sentiment classifier, a translation model, or a multi-billion-parameter chat model. Hugging Face operates the Inference API, which lets you call open models over HTTP while Hugging Face (or a partner provider routed through its platform) supplies the GPUs. It works like every hosted LLM API you have ever seen. You obtain a free access token from your account settings, you send a `POST` request whose

Authorization header carries that token as a bearer credential, the request body holds your input as JSON, and the response comes back as JSON.

7 Running Qwen Locally

Two ways to use a model:

- (1) **Hosted inference**: send an HTTP request to Hugging Face's servers (the serverless Inference API has a free tier suitable for class demos, with rate limits). Conceptually identical to Module 3 — different vendor, same bearer-token pattern, which is itself a lesson in how transferable the skills are.
- (2) **Local execution with `transformers`**: download the weights, run them in Python.

An Example: Hosted Inference (the familiar pattern)

```
import os, requests

API_URL = "https://api-inference.huggingface.co/models/distilbert-base-uncased-
    finetuned-sst-2-english"
headers = {"Authorization": f"Bearer {os.environ['HF_TOKEN']}"}

resp = requests.post(API_URL, headers=headers,
                     json={"inputs": "This course is surprisingly fun."})
print(resp.json())
```

Another Example: Local Pipeline (the new capability)

```
from transformers import pipeline

summarizer = pipeline("summarization", model="sshleifer/distilbart-cnn-12-6")
print(summarizer(long_article_text, max_length=60, min_length=20))
```